Language

🇷🇺 🇬🇧

Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Dagger 2
сделайте код чище
курс от автора сайта
StartAndroid
MADE WITH MYBANNERMAKER.COM

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 13. Modifier

24 января 2023

В этом уроке рассматриваем несколько важных моментов в работе с Modifier: порядок операторов, использование в своих Composable функциях и scope.

В одном из самых первых уроков мы уже говорили про Modifier. Мы выяснили, что с помощью Modifier мы можем настроить атрибуты, которые могут быть применимы к любому элементу. Например: размер, фон, граница. Также с помощью Modifier мы можем добавить элементу интерактивности: clickable, toggleable, scrollable, swipeable, draggable, zoomable. Полный список можно посмотреть в документации.

Чтобы правильно и эффективно использовать Modifier, необходимо обсудить несколько важных аспектов.

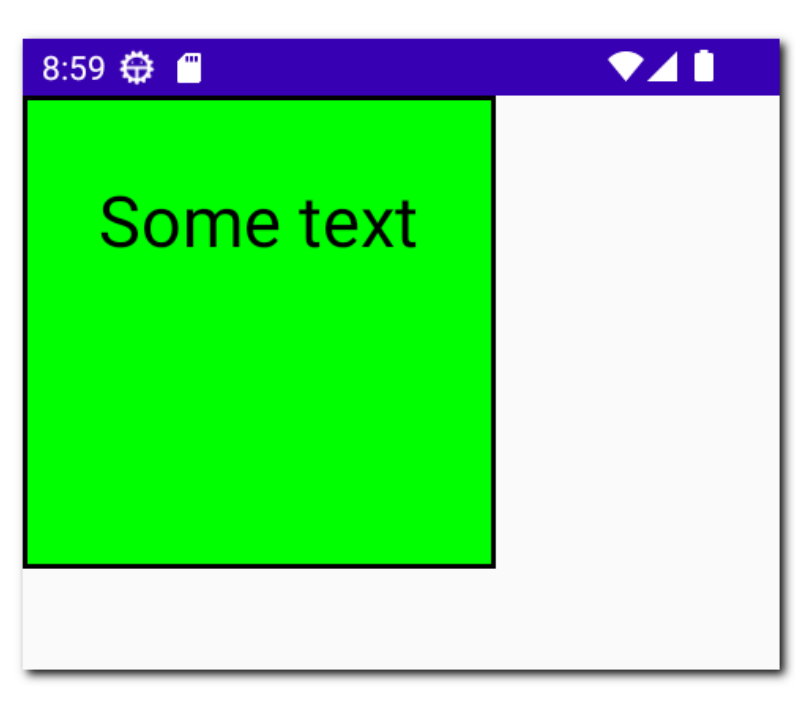
Порядок операторов

Хоть Modifier и выглядит, как классический билдер, но он не является таковым. Рассмотрим простой пример:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.size(200.dp)
            .border(2.dp, Color.Black)
            .background(Color.Green)
            .padding(32.dp)
    ) {
        Text(text = "Some text", fontSize = 30.sp)
    }
}
```

У нас есть Box размером 200, с границей, фоном и паддингом. Внутри него находится Text.

Результат:

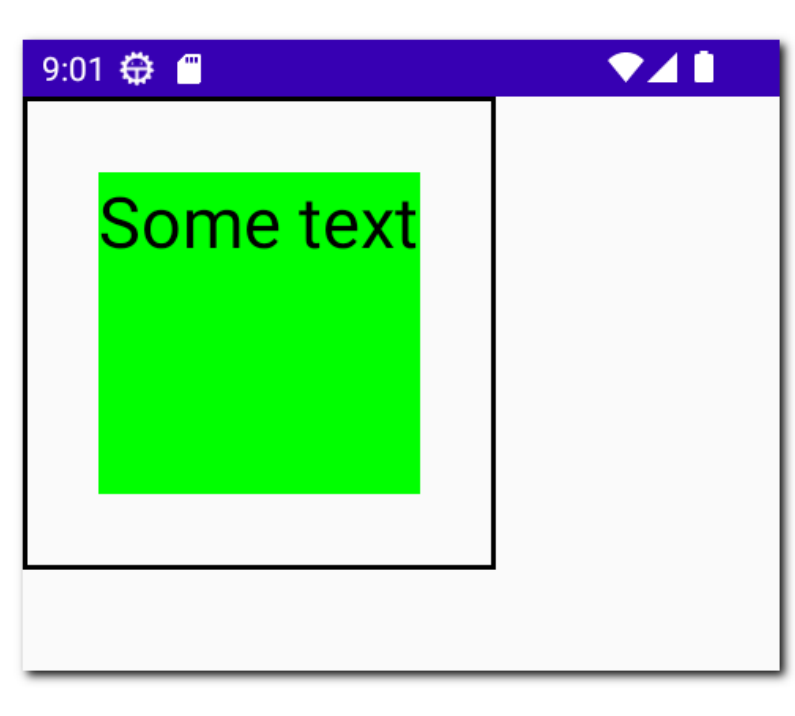


Поменяем местами операторы padding и background в Modifier:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.size(200.dp)
            .border(2.dp, Color.Black)
            .padding(32.dp)
            .background(Color.Green)
    ) {
        Text(text = "Some text", fontSize = 30.sp)
    }
}
```

Теперь padding указан перед background.

Результат:



В отличие от билдера, при работе с Modifier важен порядок операторов.

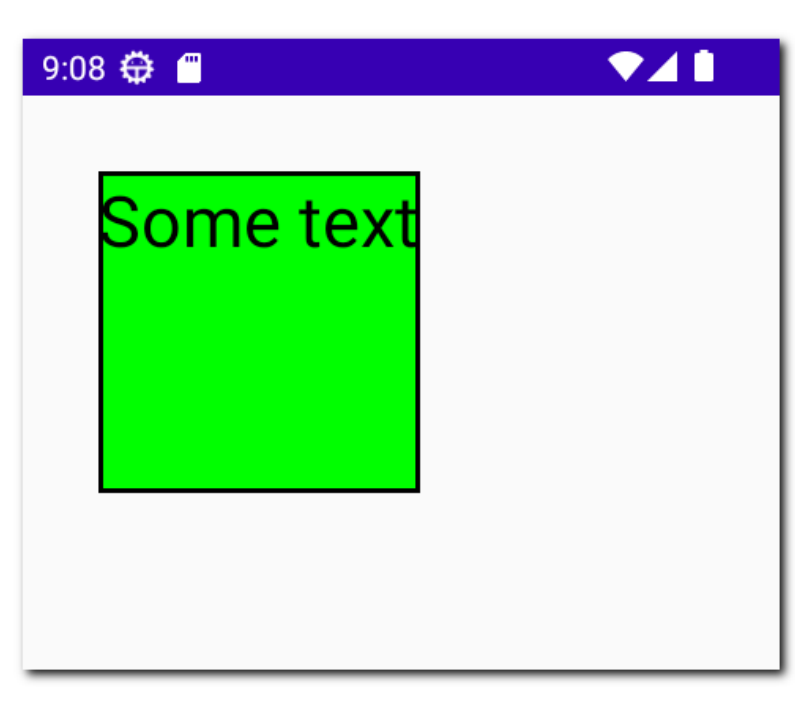
В первом примере фон был указан перед паддингом. Т.е. сначала к элементу применился фон на весь размер элемента, а потом уже добавился паддинг, который повлиял только на расположение Text внутри Box.

Во втором примере паддинг был указан перед фоном. Это привело к тому, что к элементу сначала применился паддинг, а уже к полученному результату (меньшей площади) применился фон.

Если мы поставим паддинг перед границей, то и граница у нас сместится внутрь:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.size(200.dp)
            .padding(32.dp)
            .border(2.dp, Color.Black)
            .background(Color.Green)
    ) {
        Text(text = "Some text", fontSize = 30.sp)
    }
}
```

Результат:



Это важная особенность Modifier, которую надо учитывать, чтобы получить предсказуемый результат.

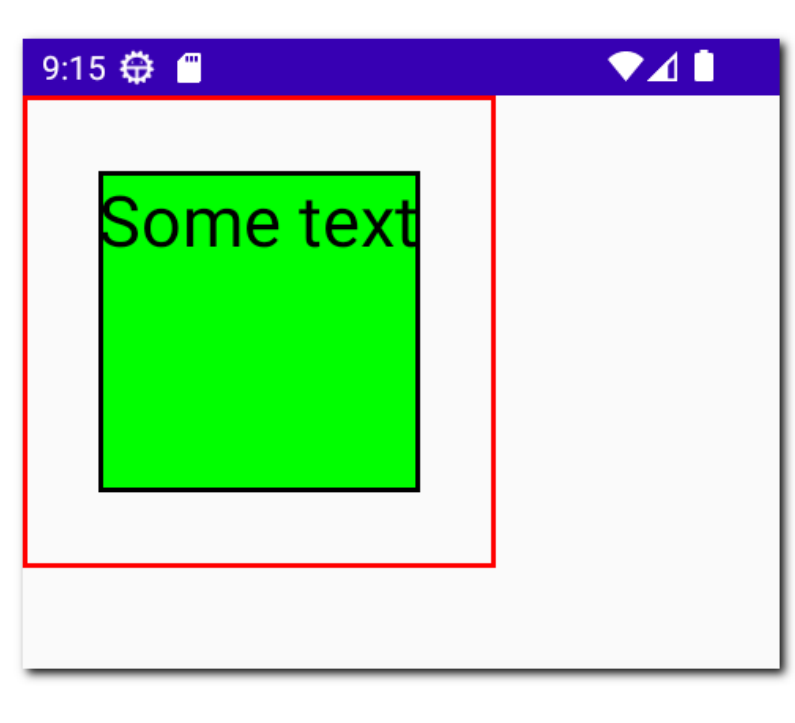
Кроме порядка есть еще одна особенность, отличающая Modifier от билдера. Мы можем один оператор использовать несколько раз. Например, мы можем сделать две границы.

Добавляем еще одну границу к прошлому примеру, в самое начало Modifier:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.size(200.dp)
            .border(2.dp, Color.Red)
            .padding(32.dp)
            .border(2.dp, Color.Black)
            .background(Color.Green)
    ) {
        Text(text = "Some text", fontSize = 30.sp)
    }
}
```

В итоге у элемента будет красная граница, соответствующая размерам элемента. А затем, после применения паддинга будет нарисована еще одна граница, черная.

Результат:



Таким образом, если вам надо на экране отобразить что-то в рамке, не спешите добавлять вложенные Box. Может быть можно обойтись силами Modifier.

Modifier параметр

Когда мы создаем свои Composable функции, рекомендуется добавлять в них параметр Modifier. Давайте посмотрим, зачем это может быть нужно.

Допустим у нас есть своя Composable функция для отображения кнопки.

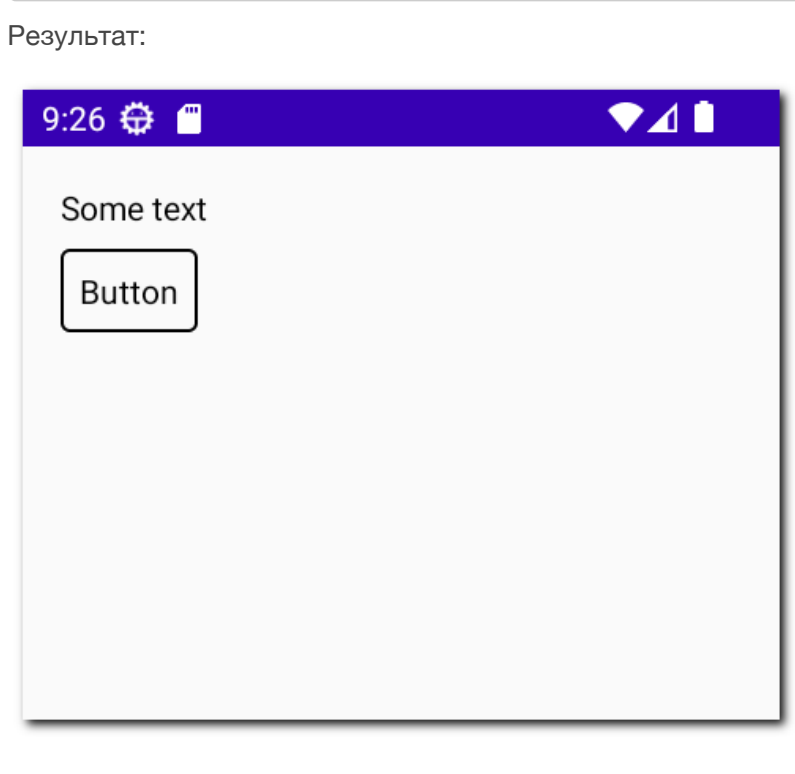
```
@Composable
fun MyButton(
    text: String,
    onClick: () -> Unit
) {
    Text(text = text,
        modifier = Modifier.clickable(onClick = onClick)
            .border(1.dp, Color.Black, RoundedCornerShape(4.dp))
            .padding(8.dp)
    )
}
```

На вход ей передаем текст, который надо отобразить, и лямбду onClick.

Добавляем эту кнопку на экран HomeScreen:

```
@Composable
fun HomeScreen() {
    Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Text(text = "Some text")
        Spacer(modifier = Modifier.height(8.dp))
        MyButton(text = "Button", onClick = { TODO() })
    }
}
```

Результат:



Мы можем эту кнопку использовать на любых экранах. Главное передавать туда текст и лямбду. По умолчанию, ширина элементов всегда определяется по содержимому, т.е. ширина нашей кнопки будет зависеть от длины текста. Но нам по дизайну на некоторых экранах необходимо иметь фиксированную ширину кнопки. Т.е. нам надо сделать так, чтобы мы при вызове MyButton могли явно задавать ширину.

Первое, что приходит в голову - это добавить в функцию MyButton новый параметр width:

```
@Composable
fun MyButton(
    text: String,
    onClick: () -> Unit,
    width: Dp
) {
    Text(text = text,
        modifier = Modifier.clickable(onClick = onClick)
            .width(width)
            .border(1.dp, Color.Black, RoundedCornerShape(4.dp))
            .padding(8.dp)
    )
}
```

Используем значение параметра width в Modifier операторе width и таким образом задаем ширину кнопки.

Вариант вроде рабочий. Мы можем снаружи функции MyButton задать ширину. Но теперь значение ширины надо указывать при каждом вызове функции. А что если надо ширину снова по содержимому сделать? Надо будет параметр width делать необязательным и добавлять проверку: если параметр задан, то используем его в Modifier. Это дополнительная логика, которой хотелось бы избежать.

Для этого мы в функцию вместо параметра width добавляем параметр modifier:

```
@Composable
fun MyButton(
    text: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Text(text = text,
        modifier = modifier.clickable(onClick = onClick)
            .border(1.dp, Color.Black, RoundedCornerShape(4.dp))
            .padding(8.dp)
    )
}
```

и внутри кнопки уже используем его в качестве стартового Modifier, к которому добавляем clickable.

В итоге мы в функции MyButton получаем modifier, который был настроен снаружи и дополнен внутри. Если снаружи нам modifier не передали, то в качестве значения по умолчанию мы используем пустой Modifier.

Теперь при вызове MyButton мы можем указать необходимую нам ширину:

```
@Composable
fun HomeScreen() {
    Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Text(text = "Some text")
        Spacer(modifier = Modifier.height(8.dp))
        MyButton(
            text = "Button", onClick = { TODO() },
            modifier = Modifier.width(100.dp)
        )
    }
}
```

Или ширину и горизонтальное выравнивание

```
@Composable
fun HomeScreen() {
    Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Text(text = "Some text")
        Spacer(modifier = Modifier.height(8.dp))
        MyButton(
            text = "Button", onClick = { TODO() },
            modifier = Modifier.width(100.dp).align(CenterHorizontally)
        )
    }
}
```

Либо можем ничего не указывать, если нас устраивают значения по умолчанию:

```
@Composable
fun HomeScreen() {
   Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
       Text(text = "Some text")
       Spacer(modifier = Modifier.height(8.dp))
       MyButton(
           text = "Button", onClick = { TODO() }
       )
   }
}
```

В примере выше функция MyButton содержит внутри себя всего один элемент - Text. Если мы создаем более сложную функцию, то modifier применяется к корневому элементу.

Скорее всего это будет один из layout: Column, Row, Box и т.п:

```
@Composable
fun MyComposable(
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier.operatorX.operatorY.operatorZ
    ) {
        // ...
    }
}
```

Также есть рекомендация - ставить modifier первым необязательным параметром функции.

В качестве примера можно взять исходники функции Text:

```
@Composable
fun Text(
    text: String,
    modifier: Modifier = Modifier,
    color: Color = Color.Unspecified,
    fontSize: TextUnit = TextUnit.Unspecified,
    fontStyle: FontStyle? = null,
    fontWeight: FontWeight? = null,
    …
) {
```

modifier тут первый необязательный параметр функции.

По своему опыту могу сказать, что добавление modifier в свои Composable функции хоть и рекомендуется, но не особо приживается в реальном коде. Поначалу вроде все пытаются делать как надо, но быстро понимают, что это практически не используется. В итоге этот параметр добавляют только в те функции, где он реально нужен.

scope

Не все операторы Modifier можно использовать в любом месте кода, есть некоторые ограничения, которые называются scope.

В качестве примера рассмотрим Column, в котором Text выравнен по центру по горизонтали:

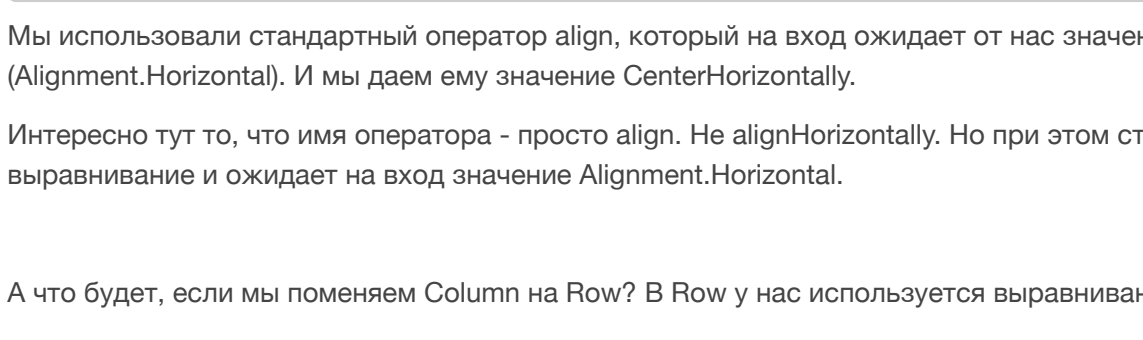
```
import androidx.compose.ui.Alignment.Companion.CenterHorizontally

@Composable
fun HomeScreen() {
    Column(modifier = Modifier.fillMaxWidth()) {
        Text(text = "Some text", modifier = Modifier.align(CenterHorizontally))
    }
}
```

Мы использовали стандартный оператор align, который на вход ожидает от нас значение горизонтального выравнивания (Alignment.Horizontal). И мы даем ему значение CenterHorizontally.

Интересно тут то, что имя оператора - просто align. Не alignHorizontally. Но при этом студия знает, что это горизонтальное выравнивание и ожидает на вход значение Alignment.Horizontal.

А что будет, если мы поменяем Column на Row? В Row у нас используется выравнивание не по горизонтали, а по вертикали.

Меняем и видим ошибку:



```
@Composable
fun HomeScreen() {
    Row(modifier = Modifier.fillMaxWidth()) { this: RowScope
        Text(text = "Some text", modifier = Modifier.align(CenterHorizontally))
    }
}
```

Type mismatch.
Required: Alignment.Vertical
Found: Alignment.Horizontal

Студия нам сообщает, что мы пытаемся значение горизонтального выравнивания подставить вместо вертикального. Т.е. теперь метод align ожидает от нас не Alignment.Horizontal, а Alignment.Vertical.

Смена Column на Row привела к тому, что оператор align(Alignment.Horizontal) был заменем оператором align(Alignment.Vertical), хотя мы его вообще не трогали. Как работает эта магия?

Вернем в наш код Column, чтобы убрать ошибку, и провалимся в исходники align:

```
fun Modifier.align(alignment: Alignment.Horizontal): Modifier
```

Видим, что это обычное Extension для Modifier. А вот его расположение не совсем обычно. Оно находится в интерфейсе ColumnScope. Это значит, что такое расширение может быть использовано только в коде, который является реализацией интерфейса ColumnScope. Или в блоке кода, у которого receiver - это ColumnScope:

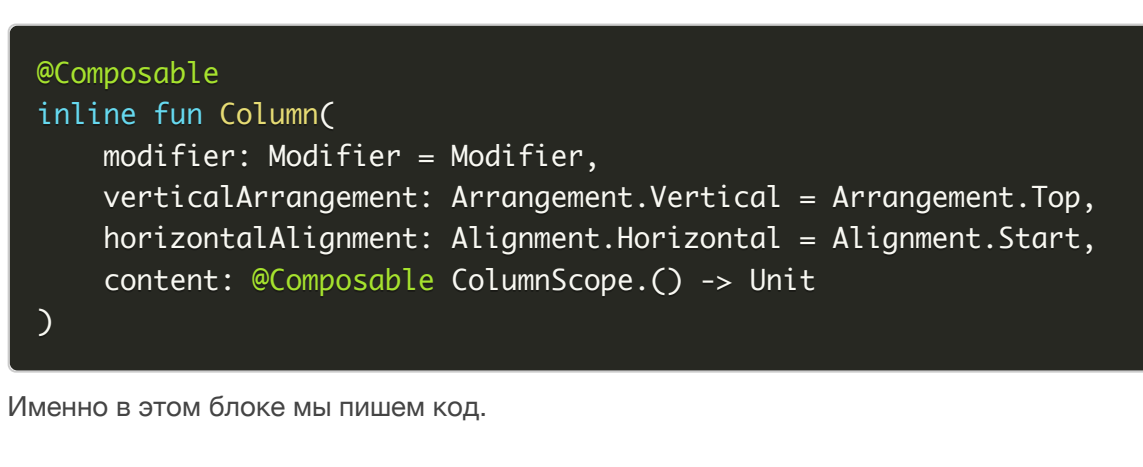
```
func: ColumnScope.() -> Unit
```

Смотрим исходники Column и видим там как раз такой блок кода content:

```
@Composable
inline fun Column(
    modifier: Modifier = Modifier,
    verticalArrangement: Arrangement.Vertical = Arrangement.Top,
    horizontalAlignment: Alignment.Horizontal = Alignment.Start,
    content: @Composable ColumnScope.() -> Unit
)
```

Именно в этом блоке мы пишем код.

Это можно увидеть и в студии:



```
@Composable
fun HomeScreen() {
    Column(modifier = Modifier.fillMaxWidth()) { this: ColumnScope
        Text(text = "Some text", modifier = Modifier.align(CenterHorizontally))
    }
}
```

Когда мы в Column { … } пишем Modifier.align, студия видит, что мы находимся в ColumnScope и понимает, что мы хотим использовать метод align(Alignment.Horizontal) из ColumnScope.

Для Row есть аналогичный интерфейс RowScope, в котором есть свое расширение align(Alignment.Vertical). И когда мы Column поменяли на Row, студия поняла, что теперь нам нужен align из RowScope и тем самым поменяла один align на другой. После чего выдала ошибку, что входной параметр не подходит.

Таким образом один и тот же оператор может иметь разные реализации в зависимости от того, где он был использован. В списке по ссылке в начале урока указан scope для каждого оператора.

Extract

Modifier, состоящий из нескольких операторов можно сохранять в переменную, чтобы потом использовать в функциях. Это имеет смысл, например, когда у нас несколько функций подряд используют один и тот же набор Modifier операторов.

Или это может пригодиться при работе с анимацией. Например, есть такая функция:

```
@Composable
fun SomeAnimatedUi(...) {
    Column(
        modifier = Modifier.fillMaxWidth()
            .background(Color.Gray)
            .padding(16.dp)
    ) {
        ...
    }
}
```

Она участвует в анимации, а значит может быть вызвана десятки раз за секунду. И каждый вызов будет заново создавать Modifier.

Этого можно избежать, сохранив Modifier в переменную вне этой функции:

```
val someModifier = Modifier.fillMaxWidth()
    .background(Color.Gray)
    .padding(16.dp)

@Composable
fun SomeAnimatedUi(...) {
    Column(
        modifier = someModifier
    ) {
        ...
    }
}
```

Теперь при перезапуске функция будет всегда использовать один и тот же экземпляр Modifier

Чтобы соединять готовые Modifier наборы друг с другом, можно использовать оператор then:

```
Column(modifier = modifier1.then(modifier2)) { … }
```

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **Опечатка** — **Евгений** 10.02.2023 17:25
Кажется опечатка - "Теперь при вызове MyText мы можем указать необходимую нам ширину:". Наверное имелось в виду myButton
Ответить

# **Опечатка** — **Denis** 13.02.2023 17:57
Кажется "scope" надо сделать заголовком блоком (жирнее)
Ответить

# **-** — **Denis** 13.02.2023 17:59
вроде ок
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.